

# DOCUMENTAÇÃO TÉCNICA - LUNARA BIOSYNC V2

---

Sistema de Análise Integrativa 360º com Base em Bioressonância Quântica

---

## 1. VISÃO GERAL DO SISTEMA

---

### Nome do Sistema

**Lunara BioSync V2** - Plataforma de Análise de Bioressonância Quântica com Integração de Inteligência Artificial

### Objetivo Principal

Sistema web para terapeutas holísticos que processa laudos de bioressonância em formato PDF utilizando IA (Google Gemini), gerando análises integrativas 360º com recomendações terapêuticas personalizadas baseadas em protocolos de Reiki (Johnny de Carli), Biomagnetismo, Auriculoterapia e Frequências Solfeggio.

### Público-Alvo

- **Primário:** Terapeutas holísticos integradores (Celso Biffe)
- **Secundário:** Profissionais de saúde alternativa
- **Usuário Final:** Clientes/pacientes que recebem os relatórios

### Diferencial

**Análise Integrativa 360º** que correlaciona:

- 6 módulos sistêmicos (Metabolismo, Energia, Inflamação, Sono, Emocional, Performance)
  - Conexão físico-emocional automática via IA
  - Histórico evolutivo com comparação temporal
  - Feedback de treino para refinamento de recomendações
  - Geração de protocolos terapêuticos com frequências Solfeggio
  - Mapeamento visual de desequilíbrios com severidade
  - Sugestão de tipo de treino (Regenerativo vs Alta Performance)
-

## 2. ARQUITETURA

### 2.1 Frontend

#### Tecnologias

```
{
  "framework": "React 19.0.0",
  "buildTool": "Vite 6.2.0",
  "linguagem": "TypeScript 5.8.2",
  "styling": "Tailwind CSS 4.1.14",
  "icons": "Lucide React 0.546.0",
  "charts": "Recharts 3.8.0",
  "routing": "React Router DOM 7.13.2",
  "animation": "Motion 12.23.24",
  "pdfGen": ["html2canvas 1.4.1", "jsPDF 4.2.1"],
  "fileUpload": "React Dropzone 15.0.0",
  "utilities": "date-fns 4.1.0"
}
```

#### Componentes Principais

```
src/
├── components/
│   ├── Dashboard.tsx           # Orquestrador principal de fluxo
│   ├── SelecaoCliente.tsx      # Seleção/criação de cliente
│   ├── Dropzone.tsx           # Upload de PDF com drag & drop
│   ├── Processing.tsx         # Tela de loading durante análise IA
│   ├── EditorRelatorio.tsx     # Edição de síntese/recomendações + preview PDF
│   ├── RelatorioFinalPDF.tsx   # Template de PDF com formatação A4
│   ├── GraficoEvolucao.tsx     # Gráfico de linha temporal (BioScore)
│   ├── DashboardEvolucao.tsx   # Comparação entre última e penúltima análise
│   └── TesteIntegracao.tsx     # Debug de conexão com Firebase/Gemini
├── services/
│   ├── aiService.ts           # Proxy para chamada backend Gemini
│   ├── firebaseService.ts     # CRUD Firestore (clientes, análises, resultados)
│   └── terapiaService.ts       # Lógica de protocolos (Reiki, Biomagnetismo, Solfeg-
gio)
├── hooks/
│   └── useExames.ts           # Hook de processamento de exame (orquestra IA + BD)
├── lib/
│   └── firebase.ts            # Inicialização Firebase (Auth, Firestore, Storage)
├── types.ts                  # Tipagens TypeScript completas
├── App.tsx                   # Autenticação Google + roteamento
└── main.tsx                  # Entry point
```

#### Fluxo de Telas (Estado da View)

```
type ViewState = 'selecao' | 'upload' | 'processando' | 'editor';

// 1. SELEÇÃO → Escolha de cliente existente ou criação de novo
// 2. UPLOAD → Dropzone para PDF + input de "Feedback de Treino" (opcional)
// 3. PROCESSANDO → Spinner enquanto IA processa
// 4. EDITOR → Preview PDF em escala (esquerda) + Editor de síntese/recomendações (direita)
```

## Estrutura de Pastas

|                 |                                             |
|-----------------|---------------------------------------------|
| /home/ubuntu/   |                                             |
| src/            | # Código-fonte React                        |
| public/         | # Assets estáticos (Logo.jpeg, favicon.png) |
| server.ts       | # Backend Node.js/Express                   |
| package.json    | # Dependências                              |
| vite.config.ts  | # Configuração Vite                         |
| tsconfig.json   | # Configuração TypeScript                   |
| firebase-*.json | # Configurações Firebase                    |
| *.rules         | # Regras de segurança Firebase              |

## 2.2 Backend

### Tecnologias

```
{
  "runtime": "Node.js",
  "framework": "Express 4.21.2",
  "ai": "@google/genai 1.29.0",
  "env": "dotenv 17.2.3",
  "devRunner": "tsx 4.21.0"
}
```

### Endpoints Principais

#### 1. POST /api/ai/generate

```
// PROTEGIDO POR AUTENTICAÇÃO (x-api-key header)
// Proxy para Google Gemini AI

Request Body:
{
  "prompt": string | Array<{text: string} | {inlineData: {mimeType: string, data: string}}>,
  "model": "gemini-3-flash-preview" | "gemini-1.5-flash" (default: gemini-3-flash-preview)
}

Headers:
{
  "x-api-key": string // Deve coincidir com APP_SECRET_KEY no servidor
}

Response:
{
  "text": string // Resposta JSON da IA
}

Errors:
- 401: Chave de API ausente ou inválida
- 500: Erro na comunicação com Gemini ou GEMINI_API_KEY ausente
```

### Função de Proxy para Gemini

**Localização:** server.ts

## Lógica de Validação de API Key (Multicamadas):

```
const getValidKey = () => {
  const keys = [
    { name: 'GEMINI_API_KEY', value: process.env.GEMINI_API_KEY },
    { name: 'VITE_GEMINI_API_KEY', value: process.env.VITE_GEMINI_API_KEY },
    { name: 'API_KEY', value: process.env.API_KEY }
  ];

  for (const k of keys) {
    let val = k.value?.trim();
    if (val) {
      // Remove aspas extras e prefixos colados por engano
      val = val.replace(/^["']|["']$/g, '').trim();

      if (val.toUpperCase().startsWith("GEMINI_API_KEY=")) {
        val = val.substring("GEMINI_API_KEY=".length).trim();
      } else if (val.toUpperCase().startsWith("API_KEY=")) {
        val = val.substring("API_KEY=".length).trim();
      }

      // Valida se não é placeholder
      if (val !== "MY_GEMINI_API_KEY" && val !== "AIzaSy..." && val.length > 10) {
        return val;
      }
    }
  }
  return null;
};
```

## Inicialização da IA:

```
const ai = apiKey ? new GoogleGenAI({ apiKey }) : null;

// Teste de conexão no startup
ai.models.generateContent({
  model: "gemini-3-flash-preview",
  contents: "ping",
}).then(() => {
  console.log("Teste de conexão Gemini: SUCESSO");
}).catch((err) => {
  console.error("Teste de conexão Gemini: FALHA");
  if (err.message?.includes("API key not valid")) {
    console.error("A chave foi REJEITADA pelo Google.");
  }
});
```

## Controle de API Keys

### Backend (Server-Side):

- GEMINI\_API\_KEY : Chave principal do Gemini (obrigatória)
- APP\_SECRET\_KEY : Token de segurança para proteger o endpoint `/api/ai/generate`

### Frontend (Client-Side):

- VITE\_GEMINI\_API\_KEY : Validada no App.tsx (verifica se começa com "Alza")
- VITE\_APP\_SECRET\_KEY : Enviada no header `x-api-key` ao chamar o backend

### Middleware de Autenticação:

```

const autenticar = (req, res, next) => {
  const token = req.headers["x-api-key"];
  const secretKey = process.env.APP_SECRET_KEY;

  if (!secretKey) {
    return res.status(500).json({
      error: "Configuração de segurança ausente no servidor"
    });
  }

  if (!token || token !== secretKey) {
    return res.status(401).json({
      error: "Não autorizado: Chave de API inválida"
    });
  }

  next();
};

```

### 3. FLUXO COMPLETO DE DADOS

#### Passo 1: Upload do PDF

**Componente:** Dropzone.tsx

- Aceita apenas arquivos .pdf
- Limite de tamanho: configurável (padrão 10MB)
- Converte para Base64 via `FileReader.readAsDataURL()`

#### Passo 2: Envio para Backend

**Hook:** `useExames.ts` → `processarExame()`

```

// 1. Converter PDF para Base64
const base64 = await fileToBase64(file);

// 2. Criar registro de análise no Firestore
const novaAnalise = await firebaseService.addAnalise({
  cliente_id: cliente.id,
  status: 'processando',
  feedback_treino: feedbackTreino
});

// 3. Buscar histórico do cliente
const historico = await firebaseService.getHistoricoEvolucao(cliente.id);

// 4. Chamar IA
const dadosExtraídos = await aiService.processarExame(
  base64.split(',')[1], // Remove prefixo "data:application/pdf;base64,"
  cliente.id,
  cliente.sexo,
  idade,
  historico,
  feedbackTreino
);

```

### Passo 3: Montagem do Prompt

**Serviço:** `aiService.ts` → `processarExame()`

```
// Contexto de Histórico (se existir)
let historicoContext = '';
if (historico && historico.length > 0) {
  const historicoSimplificado = historico.map(h => ({
    data: new Date(h.created_at).toLocaleDateString('pt-BR'),
    bioScore: h.dados_extraidos.bioScore,
    desequilibrios: h.dados_extraidos.desequilibrios.map(d =>
      `${d.sistema} (${d.severidade})`
    )
  }));
  historicoContext = `
HISTÓRICO DO CLIENTE:
${JSON.stringify(historicoSimplificado, null, 2)}
Se houver dados de exames anteriores, inclua uma "Análise Comparativa de Evolução" na
síntese_final.`;
}

// Contexto de Feedback de Treino
let feedbackContext = '';
if (feedbackTreino) {
  feedbackContext = `
FEEDBACK DE TREINO DO USUÁRIO:
"${feedbackTreino}"
Correlacione este feedback com os dados biológicos para refinar as recomendações.`;
}

// Prompt Mestre
const prompt = `Você é o Assistente de Saúde Integrativa Celso Biffe.
Atue como um Engenheiro de Dados e Terapeuta Holístico Sênior.
Analisar este laudo de bioressonância quântica para um cliente do sexo ${sexo} de ${idade} anos.

MÓDULOS OBRIGATÓRIOS (360º):
1. METABOLISMO: Dificuldade de emagrecimento, tendência a ganho de peso.
2. ENERGIA: Nível de vitalidade e causa do cansaço.
3. INFLAMAÇÃO: Nível inflamatório e origem de dores.
4. SONO: Qualidade e profundidade.
5. EMOCIONAL: Estresse e padrão emocional.
6. PERFORMANCE: Impacto hormonal na hipertrofia/catabolismo.
7. ALERTA DE TREINO: Se fadiga for Alta -> "Regenerativo", se Baixa -> "Alta Performance".
8. FEEDBACK: Correlacione o feedback do usuário: ${feedbackContext}

REGRAS:
- Correlacione físico com emocional.
- Explique a CAUSA RAIZ.
- Associe frequências Solfeggio (174Hz, 396Hz, 528Hz, 741Hz, 852Hz) conforme os desequilíbrios.
${historicoContext}

RETORNE APENAS UM JSON:
{
  "bioScore": 0-100,
  "perfil": "string",
  "fadiga": boolean,
  "metabolismo": { "dificuldade_emagrecimento": "string", "tendencia_ganho_peso":
"string", "eficiencia_metabolica": "string" },
  "energia": { "nivel_energia": "string", "causa_cansaco": "string" },
  "inflamacao": { "nivel_inflamacao": "string", "origem_dores": "string" },
  "sono": { "qualidade_sono": "string", "profundidade_sono": "string" },
  "emocional": { "nivel_estresse": "string", "padrao_emocional": "string" },
  "performance": { "potencial_hipertrofia": "string", "qualidade_recuperacao":
```

```
"string", "eficiencia_digestiva": "string", "dica_treino": "string" },
"sintese_final": "string",
"feedback_treino": "string",
"desequilibrios": [{ "sistema": "string", "severidade": "Baixa"|"Média"|"Alta",
"descricao": "string" }]
}`;
```

## Passo 4: Envio para IA

**Endpoint:** POST /api/ai/generate

```
const response = await fetch("/api/ai/generate", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "x-api-key": import.meta.env.VITE_APP_SECRET_KEY || ""
  },
  body: JSON.stringify({
    prompt: [
      { text: prompt },
      { inlineData: { mimeType: "application/pdf", data: base64Pdf } }
    ],
    model: "gemini-3-flash-preview"
  })
});
```

## Passo 5: Retorno Estruturado JSON

**Backend:** server.ts

```
const response = await ai.models.generateContent({
  model: modelName,
  contents: contents,
});

if (!response || !response.text) {
  return res.status(500).json({ error: "A IA retornou uma resposta vazia" });
}

res.json({ text: response.text });
```

## Passo 6: Processamento no Frontend

**Serviço:** aiService.ts



```

const data = await response.json();
const text = data.text;

if (!text) throw new Error('A IA não retornou conteúdo válido.');
```

// Limpeza de Markdown (caso IA retorne com ```json)

```

const jsonStr = text.replace(/```json|```/g, '').trim();
const dadosExtraídos = JSON.parse(jsonStr) as DadosExtraídos;

// Salvar no Firestore
const novoResultado = await firebaseService.addResultado({
  analise_id: novaAnalise.id,
  categoria_relatorio: 'Geral',
  dados_extraídos: dadosExtraídos
});

// Atualizar status da análise
await firebaseService.updateAnalise(novaAnalise.id, {
  status: 'concluída'
});

```

## Passo 7: Geração de Protocolos Terapêuticos

**Serviço:** `terapiaService.ts` → `gerarPlanoTerapeutico()`

```

const terapias = gerarPlanoTerapeutico(desequilíbrios);

// Algoritmo:
// - Varre cada desequilíbrio
// - Associa frequência Solfeggio via sugerirFrequencia()
// - Mapeia para protocolos de Reiki (Johnny de Carli)
// - Gera pares de Biomagnetismo por sistema
// - Sugere Auriculoterapia para casos endócrinos

```

## Passo 8: Geração de PDF

**Componente:** `EditorRelatorio.tsx` → `handleGerarPDF()`

```
// 1. Captura do DOM com html2canvas
const canvas = await html2canvas(pdfRef.current, {
  scale: 2,
  useCORS: true,
  backgroundColor: '#ffffff',
  onclone: (clonedDoc) => {
    // SOLUÇÃO RADICAL: Substituir oklch() por HEX para evitar erro de parse
    const styleTags = clonedDoc.getElementsByTagName('style');
    for (let i = 0; i < styleTags.length; i++) {
      styleTags[i].innerHTML = styleTags[i].innerHTML.replace(/oklch\([^)]+\)/g, '#737373');
    }
  }
});

// 2. Conversão canvas → JPEG → PDF
const imgData = canvas.toDataURL('image/jpeg', 0.90);
const pdf = new jsPDF({ orientation: 'p', unit: 'mm', format: 'a4', compress: true });
const pdfWidth = pdf.internal.pageSize.getWidth();
const pdfHeight = (canvas.height * pdfWidth) / canvas.width;
pdf.addImage(imgData, 'JPEG', 0, 0, pdfWidth, pdfHeight, undefined, 'FAST');
const pdfBlob = pdf.output('blob');
```

## Passo 9: Upload no Firebase Storage

```
const storage = getStorage();
const fileName = `relatorios/${cliente.id}/${analise.id}_${Date.now()}.pdf`;
const storageRef = ref(storage, fileName);

await uploadBytes(storageRef, pdfBlob);
const downloadURL = await getDownloadURL(storageRef);

// Persistir URL no Firestore
await firebaseService.updateAnalise(analise.id, {
  pdf_final_url: downloadURL,
  status: 'concluida'
});

await firebaseService.addPlano({
  analise_id: analise.id,
  cliente_id: cliente.id,
  sugestoes_terapias: terapias,
  sintese_final: sintese,
  recomendacoes_editaveis: recomendacoes,
  pdf_url: downloadURL
});
```

## Passo 10: Download Local + Feedback

```
pdf.save(`Relatorio_${cliente.nome.replace(/\s+/g, '_')}.pdf`);
alert('Relatório gerado, salvo na nuvem e baixado com sucesso!');
```

## 4. INTEGRAÇÃO COM IA (GEMINI)

### Modelo Utilizado

**Padrão:** gemini-3-flash-preview

**Alternativo:** gemini-1.5-flash

### Estrutura Completa do Prompt

Você é o Assistente de Saúde Integrativa Celso Biffe.  
Atue como um Engenheiro de Dados e Terapeuta Holístico Sênior.  
Analisar este laudo de bioressonância quântica para um cliente do sexo [SEXO] de [IDADE] anos.

MÓDULOS OBRIGATÓRIOS (360º) 📋:

1. **METABOLISMO**: Dificuldade de emagrecimento, tendência a ganho de peso.
2. **ENERGIA**: Nível de vitalidade e causa do cansaço.
3. **INFLAMAÇÃO**: Nível inflamatório e origem de dores.
4. **SONO**: Qualidade e profundidade.
5. **EMOCIONAL**: Estresse e padrão emocional.
6. **PERFORMANCE**: Impacto hormonal na hipertrofia/catabolismo.
7. **ALERTA DE TREINO**: Se fadiga **for** Alta -> "Regenerativo", se Baixa -> "Alta Performance".
8. **FEEDBACK**: Correlacione o feedback do **usuário**: [FEEDBACK\_CONTEXT]

**REGRAS:**

- Correlacione físico com emocional.
  - Explique a CAUSA RAIZ.
  - Associe frequências Solfeggio (174Hz, 396Hz, 528Hz, 741Hz, 852Hz) conforme os desequilíbrios.
- [HISTORICO\_CONTEXT]

RETORNE APENAS UM **JSON**:

🔗...estrutura completa...🔗

### Uso de Contexto

#### 1. Sexo e Idade:

```
const idade = calcularIdade(cliente.data_nascimento);
// Usado para personalizar análise hormonal e metabólica
```

#### 2. Histórico de Exames Anteriores:

```
const historico = await firebaseService.getHistoricoEvolucao(cliente.id);
// Permite análise comparativa de evolução
// IA recebe: [{data, bioScore, desequilibrios}]
```

#### 3. Feedback de Treino:

```
// Capturado em textarea opcional no Dashboard
// Exemplos: "Senti muita fadiga hoje", "Energia boa, treino intenso"
// IA correlaciona com dados biológicos para refinar dica_treino
```

## Parâmetros de Configuração

Request ao Gemini:

```
{
  model: "gemini-3-flash-preview",
  contents: {
    parts: [
      { text: prompt },
      { inlineData: { mimeType: "application/pdf", data: base64Pdf } }
    ]
  }
}
```

### Configurações de Segurança:

- Validação de formato de API Key (deve começar com "Alza")
- Múltiplos pontos de fallback (GEMINI\_API\_KEY, VITE\_GEMINI\_API\_KEY, API\_KEY)
- Modo resiliente (retorna mock em caso de falha crítica)

## 5. MÓDULOS DE ANÁLISE DA IA

### 5.1 Metabolismo

#### Campos:

- `difficuldade_emagrecimento` : string (Ex: "Alta devido a resistência insulínica")
- `tendencia_ganho_peso` : string (Ex: "Moderada, pré-diabético")
- `eficiencia_metabolica` : string (Ex: "Baixa, metabolismo lento")

**Interpretação:** Avalia taxa metabólica basal, sensibilidade insulínica, função tireoidiana.

### 5.2 Energia

#### Campos:

- `nivel_energia` : string (Ex: "Baixo", "Moderado", "Alto")
- `causa_cansaco` : string (Ex: "Fadiga adrenal por sobrecarga de cortisol")

**Interpretação:** Correlaciona com eixo HPA (hipotálamo-pituitária-adrenal), mitocôndrias, vitamina B12.

### 5.3 Inflamação

#### Campos:

- `nivel_inflamacao` : string (Ex: "Alto", "Moderado", "Baixo")
- `origem_dores` : string (Ex: "Inflamação intestinal crônica")

**Interpretação:** Biomarcadores inflamatórios (PCR, IL-6), permeabilidade intestinal.

### 5.4 Sono

#### Campos:

- `qualidade_sono` : string (Ex: "Irregular", "Boa", "Ruim")
- `profundidade_sono` : string (Ex: "Sono superficial, não alcança fase REM")

**Interpretação:** Produção de melatonina, ritmo circadiano, sistema nervoso parassimpático.

## 5.5 Emocional

### Campos:

- `nivel_estresse` : string (Ex: "Alto", "Crônico", "Moderado")
- `padrao_emocional` : string (Ex: "Ansiedade com tendência a somatização")

**Interpretação:** Correlação corpo-mente, neurotransmissores (serotonina, dopamina), sistema límbico.

## 5.6 Performance

### Campos:

- `potencial_hipertrofia` : string (Ex: "Médio, hormônios androgênicos baixos")
- `qualidade_recuperacao` : string (Ex: "Baixa, cortisol elevado")
- `eficiencia_digestiva` : string (Ex: "Comprometida por disbiose")
- `dica_treino` : string (Ex: "Treino Regenerativo: priorizando mobilidade e respiração")

### Interpretação:

- Hormônios anabólicos (testosterona, GH)
  - Relação cortisol/testosterona
  - Microbiota intestinal
  - **LÓGICA DE TREINO:**
  - Se `fadiga === true` → "Treino Regenerativo"
  - Se `fadiga === false` → "Alta Performance"
-

## 6. ESTRUTURA DO JSON DE RESPOSTA

---

### Tipagem TypeScript Completa

```

export interface DadosExtraidos {
  bioScore: number; // 0-100
  perfil: string; // Ex: "Perfil Inflamatório com Desequilíbrio Hormonal"
  fadiga: boolean; // true se fadiga adrenal detectada
  metabolismo: {
    dificuldade_emagrecimento: string;
    tendencia_ganho_peso: string;
    eficiencia_metabolica: string;
  };
  energia: {
    nivel_energia: string;
    causa_cansaco: string;
  };
  inflamacao: {
    nivel_inflamacao: string;
    origem_dores: string;
  };
  sono: {
    qualidade_sono: string;
    profundidade_sono: string;
  };
  emocional: {
    nivel_estresse: string;
    padrao_emocional: string;
  };
  performance: {
    potencial_hipertrofia: string;
    qualidade_recuperacao: string;
    eficiencia_digestiva: string;
    dica_treino: string; // Treino Regenerativo ou Alta Performance
  };
  sintese_final: string; // Síntese narrativa integrativa
  feedback_treino?: string; // Repetição do feedback do usuário
  desequilibrios: Desequilibrio[]; // Array de sistemas desequilibrados
}

export interface Desequilibrio {
  sistema: string; // Ex: "Digestivo", "Hormonal", "Nervoso"
  severidade: 'Baixa' | 'Média' | 'Alta';
  descricao: string; // Descrição detalhada do desequilíbrio
}

export interface Cliente {
  id: string;
  nome: string;
  data_nascimento: string; // YYYY-MM-DD
  whatsapp: string;
  email: string;
  sexo: 'Masculino' | 'Feminino';
  created_at: string;
}

export interface Analise {
  id: string;
  cliente_id: string;
  status: 'processando' | 'concluida' | 'erro';
  arquivo_url?: string; // URL do PDF original (se armazenado)
  pdf_final_url?: string; // URL do PDF gerado
  tipo_cliente?: string;
  feedback_treino?: string;
  sintese_vibracional?: string;
}

```

```

    created_at: string;
  }

  export interface ResultadoBioressonancia {
    id: string;
    analise_id: string;
    cliente_id?: string;
    categoria_relatorio: string;
    dados_extraidos: DadosExtraidos; // No Firestore: JSON.stringify
    created_at: string;
  }

  export interface PlanoTerapeutico {
    id: string;
    analise_id: string;
    cliente_id: string;
    sugestoes_terapias: SugestaoTerapia[]; // No Firestore: JSON.stringify
    sintese_final: string;
    recomendacoes_editaveis: string;
    pdf_url?: string;
    created_at: string;
  }

  export interface SugestaoTerapia {
    tipo: string; // Ex: "Reiki", "Biomagnetismo"
    nome?: string;
    descricao: string; // O Porquê
    resultado_esperado?: string;
    importancia?: string; // "Prioridade Máxima" | "Manutenção"
    pontos?: string[];
    frequencia?: string; // Ex: "528Hz - Milagre / Reparo de DNA"
    protocolo_detalhado?: string; // Instruções específicas
  }

  export interface FrequenciaSolfeggio {
    hz: number;
    nome: string;
    beneficio: string;
    url: string; // YouTube link
    categoria: 'fisico' | 'emocional' | 'espiritual';
  }

```

## Campos Obrigatórios (Validação)

### Firestore Rules:

- **clientes:** ['nome', 'data\_nascimento', 'sexo', 'uid']
- **analises:** ['cliente\_id', 'status', 'uid']
- **resultados\_bioressonancia:** ['analise\_id', 'dados\_extraidos', 'uid']
- **planos\_terapeuticos:** ['analise\_id', 'cliente\_id', 'uid']

## 7. LÓGICA TERAPÊUTICA

### Como Desequilíbrios são Interpretados

#### Algoritmo de Severidade:



```
const severityToNumber = (sev: string) => {
  if (sev === 'Alta') return 3;
  if (sev === 'Média') return 2;
  if (sev === 'Baixa') return 1;
  return 0;
};

// Comparação temporal para detectar piora
if (severityToNumber(currItem.severidade) > severityToNumber(prevItem.severidade)) {
  sistemasPioraram.push(currItem.sistema);
}
```

## Correlação Físico-Emocional

### Prompt Instruído:

“Correlacione físico com emocional. Explique a CAUSA RAIZ.”

### Exemplo de Saída:

“A análise 360º sugere uma correlação direta entre o sistema Digestivo (inflamação intestinal) e o padrão emocional Ansiedade Crônica. A disbiose pode estar gerando neuroinflamação via eixo intestino-cérebro, perpetuando o ciclo de estresse.”

## Uso das Frequências Solfeggio

**Função:** sugerirFrequencia(termo: string)

```
export const FREQUENCIAS_SOLFEGGIO: Record<number, FrequenciaSolfeggio> = {
  174: { hz: 174, nome: "Fundação", beneficio: "Redução de dor física e estresse",
    url: "...", categoria: 'fisico' },
  285: { hz: 285, nome: "Cognição Quântica", beneficio: "Cura de tecidos e órgãos", url: "...", categoria: 'fisico' },
  396: { hz: 396, nome: "Liberação de Medo", beneficio: "Liberação de culpa, medo e ansiedade", url: "...", categoria: 'emocional' },
  417: { hz: 417, nome: "Facilitação de Mudança", beneficio: "Limpeza de experiências traumáticas", url: "...", categoria: 'emocional' },
  528: { hz: 528, nome: "Milagre / Reparo de DNA", beneficio: "Reparação de DNA, transformação", url: "...", categoria: 'espiritual' },
  639: { hz: 639, nome: "Conexão / Relacionamentos", beneficio: "Harmonização de relacionamentos", url: "...", categoria: 'emocional' },
  741: { hz: 741, nome: "Despertar da Intuição", beneficio: "Limpeza de toxinas", url: "...", categoria: 'espiritual' },
  852: { hz: 852, nome: "Retorno à Ordem Espiritual", beneficio: "Despertar da consciência", url: "...", categoria: 'espiritual' },
  963: { hz: 963, nome: "Frequência de Deus", beneficio: "Conexão com a fonte divina", url: "...", categoria: 'espiritual' }
};

// Lógica de Associação
if (termo.includes('dor') || termo.includes('físico') || termo.includes('inflamação'))
  return FREQUENCIAS_SOLFEGGIO[174];
if (termo.includes('toxina') || termo.includes('vírus') || termo.includes('hepático'))
  return FREQUENCIAS_SOLFEGGIO[741];
// ... (fallback: 528Hz)
```

## Construção da Síntese Final

### Componentes:

1. **BioScore**: Nota de 0-100
2. **Perfil Dominante**: Ex: "Perfil Inflamatório"
3. **Desequilíbrios Detectados**: Lista com severidade
4. **Correlação Físico-Emocional**: Eixo causa-efeito
5. **Análise Comparativa** (se houver histórico): Evolução temporal
6. **Insight Terapêutico**: Recomendação de protocolo prioritário

## Influência do Histórico

Se `historico.length > 0`:

- IA recebe array simplificado de exames anteriores
- Compara `bioScore` atual vs anterior
- Identifica sistemas que melhoraram ou pioraram
- Adiciona seção "Evolução Temporal" na síntese

### Exemplo:

```
{
  "síntese_final": "Paciente apresentou melhora de 15 pontos no BioScore (72 → 87) em relação ao exame de 3 meses atrás. Sistema Digestivo evoluiu de Severidade Alta para Média, indicando resposta positiva ao protocolo de Biomagnetismo. No entanto, Sistema Emocional mantém padrão de Ansiedade, sugerindo necessidade de suporte em frequências 396Hz e 417Hz."
}
```

## 8. FIREBASE (FIRESTORE + STORAGE)

### Configuração Firebase

Arquivo: `firebase-applet-config.json`

```
{
  "projectId": "gen-lang-client-0809905480",
  "appId": "1:668673926948:web:85d3f7516604a67accb5b1",
  "apiKey": "AIzaSyCBWoMTR_rWs28vewzw08R0eG0p501W92k",
  "authDomain": "gen-lang-client-0809905480.firebaseio.com",
  "firestoreDatabaseId": "ai-studio-c3c9be34-b6c5-4245-8997-41ee5cff00cc",
  "storageBucket": "gen-lang-client-0809905480.firebaseio.com",
  "messagingSenderId": "668673926948",
  "measurementId": ""
}
```

## Estrutura de Banco de Dados (Firestore)

### Collections e Schema

1. `/clientes/{clienteId}`

```
{
  nome: string;
  data_nascimento: string; // YYYY-MM-DD
  whatsapp: string;
  email: string;
  sexo: "Masculino" | "Feminino";
  uid: string; // Firebase Auth UID (isolamento por terapeuta)
  created_at: string; // ISO 8601
}
```

## 2. /analises/{analiseId}

```
{
  cliente_id: string; // FK para /clientes
  status: "processando" | "concluida" | "erro";
  arquivo_url?: string;
  pdf_final_url?: string; // URL do PDF gerado
  tipo_cliente?: string;
  feedback_treino?: string;
  sintese_vibracional?: string;
  uid: string;
  created_at: string;
}
```

## 3. /resultados\_bioressonancia/{resultadoId}

```
{
  analise_id: string; // FK para /analises
  cliente_id?: string;
  categoria_relatorio: string;
  dados_extraidos: string; // JSON.stringify(DadosExtraidos)
  uid: string;
  created_at: string;
}
```

## 4. /planos\_terapeuticos/{planoId}

```
{
  analise_id: string;
  cliente_id: string;
  sugestoes_terapias: string; // JSON.stringify(SugestaoTerapia[])
  sintese_final: string;
  recomendacoes_editaveis: string;
  pdf_url?: string;
  uid: string;
  created_at: string;
}
```

## Relacionamentos

```
Cliente (1) ----< (N) Analise
Analise (1) ---- (1) ResultadoBioressonancia
Analise (1) ---- (1) PlanoTerapeutico
Cliente (1) ----< (N) ResultadoBioressonancia // Para histórico
```

## Firestore Storage

### Estrutura de Pastas:

```
/relatorios/  
  /{clienteId}/  
    /{analiseId}_{timestamp}.pdf
```

### Upload:

```
const storage = getStorage();  
const fileName = `relatorios/${cliente.id}/${analise.id}_${Date.now()}.pdf`;   
const storageRef = ref(storage, fileName);  
await uploadBytes(storageRef, pdfBlob);  
const downloadURL = await getDownloadURL(storageRef);
```

### URLs Públicas:

- Formato: `https://firebasestorage.googleapis.com/v0/b/{bucket}/o/{path}?alt=media&token={token}`
- Geradas via `getDownloadURL()`

## Regras de Segurança

### Firestore Rules ( `firestore.rules` )

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isAuthenticated() {
      return request.auth != null;
    }

    function isDocOwner() {
      return isAuthenticated() && request.auth.uid == resource.data.uid;
    }

    function uidUnchanged() {
      return !('uid' in request.resource.data) ||
        request.resource.data.uid == request.auth.uid;
    }

    function uidNotModified() {
      return !('uid' in request.resource.data) ||
        request.resource.data.uid == resource.data.uid;
    }

    function hasRequiredFields(fields) {
      return request.resource.data.keys().hasAll(fields);
    }

    match /clientes/{clienteId} {
      allow read: if isDocOwner();
      allow create: if isAuthenticated() && uidUnchanged() &&
        hasRequiredFields(['nome', 'data_nascimento', 'sexo', 'uid']);
      allow update: if isDocOwner() && uidNotModified();
      allow delete: if isDocOwner();
    }

    // Padrão idêntico para /analises, /resultados_bioressonancia, /pla-
    nos_terapeuticos
  }
}
```

#### Princípios:

- **Autenticação Obrigatória:** Nenhuma operação sem Firebase Auth
- **Isolamento por UID:** Cada terapeuta vê apenas seus dados
- **Imutabilidade do UID:** Não permite alteração após criação
- **Validação de Campos:** Campos obrigatórios checados no write

### Storage Rules ( `storage.rules` )

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /{allPaths=**} {
      allow read, write: if true; // ⚠️ ATENÇÃO: Totalmente aberto (melhorar em
      produção)
    }
  }
}
```

⚠ **PROBLEMA DE SEGURANÇA:** Storage está aberto. Recomendação:

```
match /relatorios/{clienteId}/{fileName} {  
  allow read: if request.auth != null;  
  allow write: if request.auth != null;  
}
```

## CORS

**Arquivo:** cors.json

```
[  
  {  
    "origin": ["*"],  
    "method": ["GET", "POST", "PUT"],  
    "responseHeader": ["Content-Type", "Authorization"],  
    "maxAgeSeconds": 3600  
  }  
]
```

**Aplicação:** Configurado no Firebase Storage para permitir acesso cross-origin.

---

## 9. GERAÇÃO DE PDF

### Bibliotecas Utilizadas

#### 1. html2canvas (1.4.1)

- Converte DOM em canvas
- Configuração: `scale: 2` para alta resolução

#### 2. jsPDF (4.2.1)

- Cria documento PDF a partir de imagem
- Formato: A4 (210mm x 297mm)

## Conversão DOM → Canvas → PDF

### Passo 1: Captura do DOM

```
const canvas = await html2canvas(pdfRef.current, {
  scale: 2, // Resolução 2x (retina)
  useCORS: true, // Permite imagens externas
  logging: false,
  backgroundColor: '#ffffff',
  onclone: (clonedDoc) => {
    const clonedElement = clonedDoc.getElementById('pdf-content');

    // Reset de transformações (importante para preview em escala)
    clonedElement.style.transform = 'none';
    clonedElement.style.display = 'block';
    clonedElement.style.width = '210mm';
    clonedElement.style.minHeight = '297mm';

    // SOLUÇÃO CRÍTICA: Substituir oklch() por HEX
    const styleTags = clonedDoc.getElementsByTagName('style');
    for (let i = 0; i < styleTags.length; i++) {
      const style = styleTags[i];
      style.innerHTML = style.innerHTML.replace(/oklch\([^)]+\)/g, '#737373');
    }

    // Injeção de CSS de compatibilidade
    const compatStyle = clonedDoc.createElement('style');
    compatStyle.innerHTML = `
      * {
        -webkit-print-color-adjust: exact !important;
        color-adjust: exact !important;
      }
      .bg-white { background-color: #ffffff !important; }
      .text-emerald-900 { color: #064e3b !important; }
      /* ... mapeamento completo de cores */
    `;
    clonedDoc.head.appendChild(compatStyle);
  }
});
```

### Passo 2: Conversão Canvas → JPEG

```
const imgData = canvas.toDataURL('image/jpeg', 0.90); // 90% qualidade
```

### Passo 3: Criação do PDF

```
const pdf = new jsPDF({
  orientation: 'p', // Portrait
  unit: 'mm',
  format: 'a4',
  compress: true
});

const pdfWidth = pdf.internal.pageSize.getWidth(); // 210mm
const pdfHeight = (canvas.height * pdfWidth) / canvas.width;

pdf.addImage(imgData, 'JPEG', 0, 0, pdfWidth, pdfHeight, undefined, 'FAST');
const pdfBlob = pdf.output('blob');
```

## Estrutura do Relatório (Template)

Seções do `RelatorioFinalPDF.tsx` :

1. **Header**: Logo + Título “Lunara BioSync”
2. **Dados do Paciente**: Nome, idade, sexo, data da análise
3. **Alertas de Piora** (condicional): Sistemas com deterioração
4. **BioScore + Síntese**: Grid 3 colunas (score, perfil, síntese)
5. **Módulos 360º**: Grid 2x3 (Metabolismo, Energia, Inflamação, Sono, Emocional, Performance)
6. **Dica de Treino**: Caixa destacada com recomendação de intensidade
7. **Insight Terapêutico**: Correlação físico-emocional
8. **Gráfico Atual**: Barra horizontal de desequilíbrios por sistema
9. **Gráfico de Evolução** (condicional): Linha temporal do BioScore
10. **Protocolo de Ativação**: Detalhamento de terapias (Reiki, Biomagnetismo)
11. **Plano de Ação Integrativo**: Recomendações editáveis pelo terapeuta
12. **Footer**: Disclaimer + créditos

## Preview no Editor

Técnica de Escala:

```
<div
  className="origin-top transition-transform duration-300"
  style={{
    transform: 'scale(0.5)', // Reduz visual para caber na tela
    width: '210mm',         // Mantém tamanho real para PDF
    height: '297mm'
  }}
>
  <div id="pdf-content" ref={pdfRef}>
    <RelatorioFinalPDF {...props} />
  </div>
</div>
```

**Importante:** O `scale(0.5)` é apenas visual. O `html2canvas` captura o tamanho real (210mm x 297mm).

## Problemas Conhecidos

### 1. Erro “oklch() invalid” no html2canvas

**Causa:** Tailwind CSS v4 usa sintaxe `oklch()` para cores, não suportada pelo parser do html2canvas.

**Solução:**

```
onclone: (clonedDoc) => {
  const styleTags = clonedDoc.getElementsByTagName('style');
  for (let i = 0; i < styleTags.length; i++) {
    styleTags[i].innerHTML = styleTags[i].innerHTML.replace(/oklch\([^)]+\)/g, '#737373');
  }
}
```

### 2. Imagens Bloqueadas por CORS

**Causa:** Imagens hospedadas em domínios externos sem headers CORS.



**Solução:**

```

```

**3. PDF com Múltiplas Páginas**

**Limitação Atual:** Sistema gera PDF de página única.

**Solução Futura:**

```
const pageHeight = pdf.internal.pageSize.getHeight();
let heightLeft = pdfHeight;
let position = 0;

pdf.addImage(imgData, 'JPEG', 0, position, pdfWidth, pdfHeight);
heightLeft -= pageHeight;

while (heightLeft >= 0) {
  position = heightLeft - pdfHeight;
  pdf.addPage();
  pdf.addImage(imgData, 'JPEG', 0, position, pdfWidth, pdfHeight);
  heightLeft -= pageHeight;
}
```

## 10. SEGURANÇA

### Gestão de API Keys

#### Backend (Server-Side)

**Variáveis de Ambiente** ( `.env` ou painel de Secrets):

```
GEMINI_API_KEY=AIzaSy... # OBRIGATÓRIA
APP_SECRET_KEY=...      # Token de proteção do endpoint /api/ai/generate
```

**Validação Multicamada:**

```
// 1. Tenta múltiplas variáveis
const keys = ['GEMINI_API_KEY', 'VITE_GEMINI_API_KEY', 'API_KEY'];

// 2. Limpa prefixos/aspas acidentais
val = val.replace(/^['"]|['"]$/g, '').trim();

// 3. Valida formato (deve começar com "AIza")
if (!apiKey.startsWith("AIza")) {
  console.error("Formato inválido");
}

// 4. Teste de conexão no startup
ai.models.generateContent({ model: "...", contents: "ping" });
```

## Frontend (Client-Side)

**Variáveis de Ambiente** ( `.env.local` ou build config):

```
VITE_GEMINI_API_KEY=AizaSy... # Validada no App.tsx
VITE_APP_SECRET_KEY=...      # Enviada no header x-api-key
```

**Validação no App.tsx:**

```
const apiKey = import.meta.env.VITE_GEMINI_API_KEY;
const isKeyMissing = !apiKey || apiKey === "MY_GEMINI_API_KEY" || apiKey === "AizaSy..";
const isKeyInvalid = apiKey && !apiKey.startsWith("Aiza");

if (isKeyMissing || isKeyInvalid) {
  // Exibe tela de erro com instruções
}
```

## Proteção de Endpoints

**Middleware de Autenticação:**

```
const autenticar = (req, res, next) => {
  const token = req.headers["x-api-key"];
  const secretKey = process.env.APP_SECRET_KEY;

  if (!secretKey) {
    return res.status(500).json({
      error: "APP_SECRET_KEY não definida"
    });
  }

  if (!token || token !== secretKey) {
    return res.status(401).json({
      error: "Não autorizado"
    });
  }

  next();
};

app.post("/api/ai/generate", autenticar, async (req, res) => {
  // Endpoint protegido
});
```

## Separação Frontend/Backend

**Razão:** Ocultar `GEMINI_API_KEY` do cliente.

**Arquitetura:**

```
Frontend (Browser)
  ↓ POST /api/ai/generate + x-api-key header
Backend (Node.js)
  ↓ Valida x-api-key === APP_SECRET_KEY
  ↓ Usa GEMINI_API_KEY (não exposta)
Google Gemini API
```

## Autenticação de Usuário

**Provider:** Firebase Authentication (Google OAuth)

**Fluxo:**

```
// Login
const provider = new GoogleAuthProvider();
await signInWithPopup(auth, provider);

// Listener de Estado
onAuthStateChanged(auth, (user) => {
  setUser(user);
  // user.uid usado para isolamento de dados no Firestore
});

// Logout
await signOut(auth);
```

**Isolamento de Dados:**

- Firestore Rules verificam `request.auth.uid == resource.data.uid`
- Cada terapeuta vê apenas seus clientes/análises

**Limitações Atuais:**

- ❌ Não há rate limiting no endpoint `/api/ai/generate`
- ❌ Storage está totalmente aberto (allow read, write: if true)
- ❌ Não há criptografia de campos sensíveis (whatsapp, email)

## 11. PROBLEMAS ENFRENTADOS E SOLUÇÕES

### Problema 1: CORS ao Acessar Firebase Storage

**Sintoma:** Erro “No ‘Access-Control-Allow-Origin’ header” ao carregar imagens no PDF.

**Causa:** Bucket do Firebase sem configuração CORS.

**Solução:**

```
gsutil cors set cors.json gs://gen-lang-client-0809905480.firebasestorage.app
```

```
// cors.json
[
  {
    "origin": ["*"],
    "method": ["GET", "POST", "PUT"],
    "responseHeader": ["Content-Type", "Authorization"],
    "maxAgeSeconds": 3600
  }
]
```

### Problema 2: API\_KEY\_INVALID do Gemini

**Sintoma:** Erro “API key not valid” mesmo com chave correta.

**Causa:** Chave com prefixo/sufixo acidental (ex: "AIzaSy..." com aspas).

**Solução:**

```
val = val.replace(/^["']|["']$/g, '').trim(); // Remove aspas extras

// Remove prefixos se usuário colou linha completa do .env
if (val.toUpperCase().startsWith("GEMINI_API_KEY=")) {
  val = val.substring("GEMINI_API_KEY=".length).trim();
}
```

## Problema 3: html2canvas Quebra com oklch()

**Sintoma:** Erro "Error: Expected a color" ao gerar PDF.

**Causa:** Tailwind v4 usa `oklch(...)` para cores, sintaxe não suportada pelo parser CSS do html2canvas.

**Solução:**

```
onclone: (clonedDoc) => {
  const styleTags = clonedDoc.getElementsByTagName('style');
  for (let i = 0; i < styleTags.length; i++) {
    styleTags[i].innerHTML = styleTags[i].innerHTML.replace(/oklch\([^)]+\)/g, '#737373');
  }

  // Injetar CSS de compatibilidade com HEX
  const compatStyle = clonedDoc.createElement('style');
  compatStyle.innerHTML = `
    .bg-white { background-color: #ffffff !important; }
    .text-emerald-900 { color: #064e3b !important; }
    /* ... */
  `;
  clonedDoc.head.appendChild(compatStyle);
}
```

## Problema 4: Preview do PDF Não Cabe na Tela

**Sintoma:** Template A4 (210mm x 297mm) fica gigante na tela.

**Solução:** Aplicar `transform: scale(0.5)` no container de preview, mas manter tamanho real para captura.

```
<div style={{ transform: 'scale(0.5)', width: '210mm', height: '297mm' }}>
  <div ref={pdfRef}>
    <RelatorioFinalPDF />
  </div>
</div>
```

## Problema 5: Dados JSON Muito Grandes no Firestore

**Sintoma:** `dados_extraidos` e `sugestoes_terapias` são objetos complexos.

**Solução:** Armazenar como string serializada.

```
// Write
await addDoc(collection(db, 'resultados_bioressonancia'), {
  ...resultado,
  dados_extraidos: JSON.stringify(resultado.dados_extraidos)
});

// Read
const data = doc.data();
return {
  ...data,
  dados_extraidos: JSON.parse(data.dados_extraidos)
};
```

## Problema 6: Histórico de Evolução Vazio

**Sintoma:** Gráfico de evolução não aparece para clientes com múltiplos exames.

**Causa:** Faltava adicionar `cliente_id` na collection `resultados_bioressonancia`.

**Solução:** Modificar `addResultado()` para incluir `cliente_id` implícito via `analise.cliente_id`.

## Problema 7: IA Retorna Texto com Markdown

**Sintoma:** JSON vem envolvido em ````json ... ````.

**Solução:**

```
const jsonStr = text.replace(/```json|```/g, '').trim();
const dadosExtraidos = JSON.parse(jsonStr);
```

## Problema 8: Timeout ao Processar PDFs Grandes

**Sintoma:** PDFs > 5MB causam timeout na IA.

**Solução (Implementar):**

- Comprimir PDF antes de enviar (via `pdf-lib` ou similar)
- Limitar tamanho no Dropzone (10MB)

# 12. MELHORIAS FUTURAS

## 1. Multiidioma

- Detectar idioma do cliente

- Traduzir relatórios automaticamente
- Usar `i18next` no frontend

## 2. Notificações via WhatsApp/Email

- Integrar Twilio ou WhatsApp Business API
- Enviar PDF automaticamente após geração
- Usar SendGrid ou Firebase Functions para email

## 3. Agendamento de Sessões

- Calendário integrado no sistema
- Notificações de lembretes
- Integração com Google Calendar

## 4. Dashboard de Métricas para Terapeuta

- Gráficos de evolução agregada de todos os clientes
- Identificação de padrões comuns
- Taxa de melhora vs. piora

## 5. Modo Offline

- Service Workers para cache de dados
- Sincronização quando voltar online
- Progressive Web App (PWA)

## 6. Controle de Acesso Granular

- Múltiplos níveis de permissão (Admin, Terapeuta, Assistente)
- Auditoria de alterações
- Logs de acesso

## 7. Integração com Wearables

- Importar dados de Fitbit, Apple Watch
- Correlação de métricas objetivas (frequência cardíaca, sono) com análise

## 8. Testes Automatizados

- Jest para testes unitários
- Cypress para testes E2E
- Cobertura > 80%

## 9. CI/CD

- GitHub Actions para deploy automático
- Ambientes de staging e produção
- Rollback automático em caso de erro

## 10. Versionamento de Protocolos

- Permitir terapeuta customizar protocolos de Reiki/Biomagnetismo
- Histórico de versões de frequências Solfeggio

## 11. Suporte a Múltiplos Terapeutas por Organização

- Modelo SaaS com assinatura
- Isolamento por `organizacao_id` além de `uid`

## 12. Análise de Sentimento no Feedback

- NLP para extrair emoções do feedback de treino
- Correlação automática com padrão emocional

---

## 13. GUIA DE RECONSTRUÇÃO

### 13.1 Variáveis de Ambiente Necessárias

#### Backend (Node.js):

```
# .env (raiz do projeto)
GEMINI_API_KEY=AIZA...           # Google AI Studio API Key
APP_SECRET_KEY=...               # Token de segurança (gerar com: openssl rand -hex 32)
NODE_ENV=development            # ou production
```

#### Frontend (Vite):

```
# .env.local ou .env
VITE_GEMINI_API_KEY=AIZA...       # Mesma chave do backend (validação no App.tsx)
VITE_APP_SECRET_KEY=...          # Mesma chave do APP_SECRET_KEY
```

#### Firebase:

- Configuração está hardcoded em `firebase-applet-config.json`
- ⚠ **RISCO:** Credenciais expostas no código-fonte
- **Solução:** Mover para variáveis de ambiente

```
VITE_FIREBASE_API_KEY=AIZA...
VITE_FIREBASE_PROJECT_ID=gen-lang-client-...
VITE_FIREBASE_STORAGE_BUCKET=...
VITE_FIREBASE_FIRESTORE_DB_ID=ai-studio-...
```

---

## 13.2 Comandos de Instalação

```
# 1. Clonar/Extrair o projeto
cd /home/ubuntu
unzip biosync-v2.zip
cd biosync-v2

# 2. Instalar dependências
npm install

# 3. Configurar variáveis de ambiente
cp .env.example .env
# Editar .env com suas credenciais

# 4. Instalar dependências dev (se necessário)
npm install -D typescript tsx @types/express @types/node

# 5. Verificar instalação
npm list
```

## 13.3 Configurações Obrigatórias

### 1. Firebase Console

1. Criar projeto no [Firebase Console](https://console.firebase.google.com/) (<https://console.firebase.google.com/>)
2. Habilitar:
  - **Authentication:** Google Provider
  - **Firestore Database:** Modo “production” ou “test”
  - **Storage:** Criar bucket
3. Copiar configuração para `firebase-applet-config.json`
4. Deploy de regras:

```
bash
npm install -g firebase-tools
firebase login
firebase deploy --only firestore:rules
firebase deploy --only storage:rules
```

### 2. Google AI Studio

1. Acessar [AI Studio](https://aistudio.google.com/) (<https://aistudio.google.com/>)
2. Criar API Key
3. Copiar chave (começa com “Alza”)
4. Adicionar ao `.env` como `GEMINI_API_KEY`

### 3. Configurar CORS no Firebase Storage

```
# Instalar gsutil (Google Cloud SDK)
# https://cloud.google.com/storage/docs/gsutil_install

# Aplicar CORS
gsutil cors set cors.json gs://[SEU-BUCKET].firebasestorage.app
```



## 13.4 Comandos de Execução

### Desenvolvimento

```
# Inicia servidor Node.js + Vite HMR  
npm run dev  
  
# Acesso: http://localhost:3000
```

### Produção

```
# 1. Build do frontend  
npm run build  
  
# 2. Iniciar servidor (serve frontend estático + API)  
npm start  
  
# Acesso: http://localhost:3000
```

### Debug

```
# Rota de teste de integração  
# http://localhost:3000/debug  
# Verifica conexão Firebase + Gemini
```

---

## 13.5 Estrutura de Pastas para Reconstrução

|                             |                                    |
|-----------------------------|------------------------------------|
| biosync-v2/                 |                                    |
| src/                        | # Código-fonte React               |
| components/                 | # Componentes UI                   |
| Dashboard.tsx               | # Orquestrador principal           |
| SelecaoCliente.tsx          | # CRUD de clientes                 |
| Dropzone.tsx                | # Upload de PDF                    |
| Processing.tsx              | # Tela de loading                  |
| EditorRelatorio.tsx         | # Editor + preview                 |
| RelatorioFinalPDF.tsx       | # Template PDF                     |
| GraficoEvolucao.tsx         | # Gráfico de linha                 |
| DashboardEvolucao.tsx       | # Comparativo temporal             |
| TesteIntegracao.tsx         | # Debug                            |
| services/                   | # Lógica de negócio                |
| aiService.ts                | # Proxy Gemini                     |
| firebaseService.ts          | # CRUD Firestore                   |
| terapiaService.ts           | # Protocolos terapêuticos          |
| hooks/                      |                                    |
| useExames.ts                | # Hook de processamento            |
| lib/                        |                                    |
| firebase.ts                 | # Inicialização Firebase           |
| types.ts                    | # Tipagens TypeScript              |
| App.tsx                     | # Autenticação + Routing           |
| main.tsx                    | # Entry point                      |
| index.css                   | # Estilos globais                  |
| public/                     | # Assets estáticos                 |
| Logo.jpeg                   |                                    |
| favicon.png                 |                                    |
| server.ts                   | # Backend Node.js/Express          |
| package.json                | # Dependências                     |
| vite.config.ts              | # Config Vite                      |
| tsconfig.json               | # Config TypeScript                |
| firebase-applet-config.json | # Credenciais Firebase             |
| firebase-blueprint.json     | # Schema do banco                  |
| firestore.rules             | # Regras de segurança Firestore    |
| storage.rules               | # Regras de segurança Storage      |
| cors.json                   | # Configuração CORS                |
| .env                        | # Variáveis de ambiente (backend)  |
| .env.local                  | # Variáveis de ambiente (frontend) |
| README.md                   | # Documentação básica              |

## 13.6 Checklist de Reconstrução

### Fase 1: Setup Inicial

- [ ] Instalar Node.js (v18+)
- [ ] Clonar/extrair projeto
- [ ] `npm install`
- [ ] Criar `.env` e `.env.local`

### Fase 2: Configuração Firebase

- [ ] Criar projeto no Firebase Console
- [ ] Habilitar Authentication (Google)
- [ ] Criar Firestore Database
- [ ] Criar Storage Bucket

- [ ] Copiar configuração para `firebase-applet-config.json`
- [ ] Deploy de rules: `firebase deploy --only firestore:rules,storage:rules`
- [ ] Configurar CORS: `gsutil cors set cors.json gs://[BUCKET]`

### Fase 3: Configuração Gemini

- [ ] Criar API Key no Google AI Studio
- [ ] Adicionar `GEMINI_API_KEY` ao `.env`
- [ ] Adicionar `VITE_GEMINI_API_KEY` ao `.env.local`
- [ ] Testar conexão: `npm run dev` → acessar `/debug`

### Fase 4: Segurança

- [ ] Gerar `APP_SECRET_KEY`: `openssl rand -hex 32`
- [ ] Adicionar ao `.env` (backend) e `.env.local` (frontend)
- [ ] Verificar Firestore Rules estão ativas
- [ ] **IMPORTANTE:** Restringir Storage Rules (substituir `allow read, write: if true`)

### Fase 5: Testes

- [ ] Criar conta Google de teste
- [ ] Fazer login no sistema
- [ ] Cadastrar cliente
- [ ] Upload de PDF de teste
- [ ] Verificar análise gerada
- [ ] Editar recomendações
- [ ] Gerar PDF
- [ ] Verificar upload no Firebase Storage
- [ ] Testar download do PDF

### Fase 6: Deploy (Opcional)

- [ ] Build: `npm run build`
- [ ] Deploy backend (Heroku, Railway, Google Cloud Run)
- [ ] Deploy frontend (Vercel, Netlify, Firebase Hosting)
- [ ] Configurar variáveis de ambiente na plataforma
- [ ] Testar em produção

## 13.7 Troubleshooting Comum

### Erro: “Cannot find module ‘express’”

- Solução: `npm install express @types/express`

### Erro: “API key not valid”

- Solução: Verificar se chave começa com “Alza”, sem aspas/espacos extras

### Erro: “Firebase: Firebase App named ‘[DEFAULT]’ already exists”

- Solução: Garantir que `firebase.ts` é importado apenas uma vez

### Erro: “No ‘Access-Control-Allow-Origin’ header”

- Solução: Configurar CORS no Firebase Storage (ver seção 8.6)

**Erro: “Expected a color” ao gerar PDF**

- Solução: Verificar se código de substituição oklch está ativo no `onclone`

**Erro: “Missing or insufficient permissions”**

- Solução: Verificar Firestore Rules, fazer logout/login

## 13.8 Resumo dos Endpoints

**Frontend:** `http://localhost:3000`

- `/` - Dashboard principal (requer login)
- `/debug` - Teste de integração

**Backend API:** `http://localhost:3000/api`

- `POST /api/ai/generate` - Proxy para Gemini (PROTEGIDO)

**Firebase:**

- Firestore: `ai-studio-c3c9be34-b6c5-4245-8997-41ee5cff00cc`
- Storage: `gen-lang-client-0809905480.firebaseiostorage.app`
- Auth: `gen-lang-client-0809905480.firebaseioapp.com`

## 13.9 Dependências Críticas

**Produção:**

```
{
  "@google/genai": "1.29.0",      // IA
  "express": "4.21.2",           // Backend
  "firebase": "12.11.0",         // Backend-as-a-Service
  "react": "19.0.0",             // UI
  "html2canvas": "1.4.1",        // PDF (captura)
  "jspdf": "4.2.1",              // PDF (geração)
  "recharts": "3.8.0",           // Gráficos
  "lucide-react": "0.546.0",     // Ícones
  "dotenv": "17.2.3"             // Env vars
}
```

**Desenvolvimento:**

```
{
  "typescript": "5.8.2",
  "tsx": "4.21.0",                // Execução Node.js com TS
  "@types/express": "4.17.21",
  "vite": "6.2.0",
  "tailwindcss": "4.1.14"
}
```

## 13.10 Pontos de Atenção para LLMs

1. **NÃO expor GEMINI\_API\_KEY no frontend:** Sempre usar backend como proxy
2. **Firestore Rules são CRÍTICAS:** Sem elas, dados ficam públicos

3. **oklch() quebra html2canvas**: Sempre substituir por HEX
4. **Base64 deve remover prefixo**: `base64.split(',')[1]`
5. **JSON.stringify ao salvar objetos complexos no Firestore**
6. **UID de autenticação é chave de isolamento**: Nunca permitir modificação
7. **Storage Rules estão ABERTAS**: Corrigir em produção
8. **CORS deve ser configurado manualmente no Storage**
9. **PDF preview usa scale()**: Não alterar width/height reais
10. **Feedback de treino é opcional**: Sistema funciona sem ele

---

## CONCLUSÃO

---

O **Lunara BioSync V2** é um sistema completo de análise integrativa 360º que combina:

- **Inteligência Artificial** (Google Gemini) para interpretação de laudos
- **Protocolos Terapêuticos** automatizados (Reiki, Biomagnetismo, Solfeggio)
- **Histórico Evolutivo** com comparação temporal
- **Geração de PDF** profissional com gráficos
- **Arquitetura Segura** com isolamento por terapeuta

O sistema foi projetado para ser **intuitivo para terapeutas** (não desenvolvedores), com fluxo linear e validações em todas as etapas. A separação frontend/backend garante que credenciais sensíveis (GEMINI\_API\_KEY) nunca sejam expostas ao cliente.

### Pontos Fortes:

- ✓ Análise multimodal (PDF + contexto textual)
- ✓ Correlação físico-emocional automática
- ✓ Protocolos terapêuticos embasados (Johnny de Carli)
- ✓ Histórico evolutivo com alertas de piora
- ✓ PDF profissional com branding
- ✓ Autenticação Firebase (Google OAuth)

### Pontos de Melhoria:

- ⚠ Storage sem autenticação
- ⚠ Falta de rate limiting
- ⚠ Sem testes automatizados
- ⚠ Credenciais Firebase hardcoded
- ⚠ Sem suporte a múltiplas páginas no PDF

Este documento serve como **mapa completo** para qualquer desenvolvedor ou LLM reconstruir o sistema do zero, entender sua arquitetura e evoluí-lo de forma segura.

---

**Autor Original:** Celso Biffe (Terapeuta Holístico Integrativo)

**Stack:** React 19 + TypeScript + Vite + Firebase + Google Gemini

**Versão:** 2.0

**Data:** Março de 2026

**Documentação Gerada Por:** DeepAgent (Abacus.AI)